

EP1 – Agente de Busca para o Pac-Man

Lucas Aguiar Garcia – 13672770

Resultados

Tabela 1: Comparação das buscas em grafos artificiais

Algoritmo	Instância	Nós expandidos	Tamanho do plano
DFS (Q1)	graph_backtrack	3	2
	graph_bfs_vs_dfs	2	1
	graph_infinite	3	3
	graph_manypaths	6	4
BFS (Q2)	graph_backtrack	4	2
	graph_bfs_vs_dfs	2	1
	graph_infinite	3	3
	graph_manypaths	8	4
UCS (Q3)	graph_backtrack	4	2
	graph_bfs_vs_dfs	2	1
	graph_infinite	3	3
	graph_manypaths	8	4
	ucs_0_graph	5	3
A* (Q4)	astar_0	5	3
	astar_1_graph_heuristic	4	3
	graph_backtrack	4	2
	graph_manypaths	8	4

Tabela 2: Comparação das buscas no labirinto *mediumMaze*

Algoritmo	Variante	Nós expandidos	Tamanho do plano
DFS (Q1)	mediumMaze	144	130
BFS (Q2)	mediumMaze	269	68
UCS Q3	mediumMaze (C)	269	68
UCS Q3	mediumMaze (E)	260	74
UCS Q3	mediumMaze (W)	173	152
A* Manhattan (Q4)	mediumMaze	221	68

Tabela 3: Buscas em instâncias especiais (Corners e FoodSearch)

Questão	Instância	Nós expandidos	Tamanho do plano
Q7 – CornersProblem	tinyCorner	—	28
Q8 – FoodHeuristic	(padrão)	0 (heurística ótima)	106

Discussão

Vantagens e Desvantagens

A Tabela 2 permite comparar diretamente os algoritmos no mesmo ambiente (*mediumMaze*), evidenciando o compromisso entre eficiência de busca e qualidade do plano.

DFS expandiu apenas 144 nós — o menor número entre todos os algoritmos —, porém produziu um plano de 130 passos, quase o dobro do ótimo (68 passos). Isso reflete a principal desvantagem da busca em profundidade: sem garantia de otimalidade, o caminho encontrado pode ser arbitrariamente longo. A vantagem de baixo custo computacional só é relevante quando a qualidade da solução não é crítica.

BFS encontrou o plano ótimo de 68 passos, mas ao custo de 269 nós expandidos. Por explorar todos os nós a uma dada profundidade antes de avançar, a BFS garante otimalidade em grafos com custos uniformes. A desvantagem é o uso intensivo de memória e o número elevado de expansões, que cresce exponencialmente com a profundidade da solução.

UCS apresentou comportamento idêntico ao da BFS no cenário de custo uniforme (*mediumMaze* padrão), expandindo também 269 nós com plano de 68 passos. Sua vantagem emerge em cenários com custos heterogêneos: ao ordenar a fila pela prioridade do custo acumulado $g(n)$, o UCS garante otimalidade mesmo quando as ações têm custos distintos.

A* com heurística de distância de Manhattan obteve o melhor equilíbrio: encontrou o plano ótimo de 68 passos expandindo apenas 221 nós, uma redução de 18% em relação à BFS e ao UCS.

Questões Teóricas

Questão 1

A ordem de exploração seguiu o esperado?

A DFS usa uma pilha. Os sucessores são gerados na ordem N, S, E, W, mas como são empilhados, o último inserido é o primeiro explorado. Na prática, West é explorado primeiro, depois East, South, North. Isso gera um padrão de exploração que pode parecer contra-intuitivo se comparado à ordem de geração.

O tamanho da solução está de acordo?

A DFS não garante solução ótima. Para *tinyMaze*, *mediumMaze* e *bigMaze*, o caminho encontrado pode ser longo e tortuoso, dependendo da ordem de exploração e da estrutura do labirinto. É esperado que a solução seja subótima.

O que aconteceria com DFS em árvore?

Sem controle de estados visitados, a DFS em árvore poderia entrar em ciclos infinitos (por exemplo, em corredores que permitem ir e voltar), nunca terminando. O grafo de busca do Pac-Man tem ciclos, então a completude só é garantida na versão em grafo.

Questão 2

A BFS encontra solução de custo mínimo?

Sim, quando todos os custos de ação são iguais (custo unitário, como no `PositionSearchProblem`), a BFS encontra a solução de menor número de passos, que equivale ao menor custo. Ela é ótima nesse cenário.

Se mudarmos a ordem dos sucessores, a BFS encontra soluções diferentes?

A solução encontrada pode diferir em termos de qual caminho específico é retornado (quando há empate no comprimento), mas o comprimento ótimo da solução permanece o mesmo. A BFS sempre encontra o caminho mais curto em número de passos, independentemente da ordem de geração dos sucessores.

Questão 3

O `StayEastSearchAgent` utiliza uma função de custo da forma $(0,5)^x$. Assim, posições mais à direita (maior x) possuem custo menor. Dessa forma, o agente é incentivado a permanecer no lado leste do labirinto, pois mover-se nessa direção é mais barato. Como consequência, o caminho encontrado tende a seguir pelo lado direito do tabuleiro.

Por outro lado, o `StayWestSearchAgent` utiliza uma função de custo da forma 2^x . Nesse caso, posições mais à direita tornam-se exponencialmente mais caras. O agente é fortemente penalizado por se mover para o leste, o que o leva a preferir caminhos pelo lado oeste, mesmo que esses caminhos sejam geometricamente mais longos. No ambiente *mediumScaryMaze*, que possui fantasmas e corredores mais à direita, o agente tende a contornar os obstáculos seguindo pelo lado oeste.

Em ambos os casos, o UCS encontra o caminho de menor custo total de acordo com a função de custo definida, e não necessariamente o caminho mais curto em número de passos.

Questão 4

Por que A* é mais rápido?

O algoritmo A* utiliza a função de avaliação $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo acumulado até o nó e $h(n)$ é uma heurística que estima o custo restante até o objetivo. Essa estratégia direciona a busca em direção ao objetivo, evitando a expansão de nós que estão “longe” da solução.

Diferentemente de algoritmos como BFS e UCS, que expandem os nós de forma mais uniforme em todas as direções, o A* prioriza caminhos mais promissores. Como resultado,

o número de nós expandidos tende a ser significativamente menor, tornando a busca mais eficiente.

Por que usar heurística consistente?

Uma heurística consistente (ou monotônica) satisfaz a propriedade:

$$h(n) \leq c(n, n') + h(n')$$

para todo sucessor n' de n .

Essa condição garante que, uma vez que um nó é removido da fila de prioridade, seu custo $g(n)$ já é ótimo, não sendo necessário revisitá-lo. Com isso, a busca em grafo com A^* é completa e ótima, ou seja, sempre encontra a solução de menor custo sem precisar reabrir nós já expandidos.

Por outro lado, uma heurística apenas admissível (mas não consistente) pode exigir a reabertura de nós, o que torna a implementação mais complexa e potencialmente menos eficiente.